

EMBEDDED LINUX SYSTEMS

ASSIGNMENT 01

How to Write a Device Driver

Student Name: Nguyen Dang Quang

Student ID: SE201072

Platform: ARMv7 Cortex-A9 / QEMU vexpress-a9

Linux Kernel: 5.15.0

Final Report

1. System Architecture

The assignment was implemented and tested on an embedded Linux environment running on QEMU. The system uses an ARM Cortex-A9 virtual processor, ARMv7 architecture, Linux kernel 5.15.0, an initramfs root filesystem, and the arm-linux-gnueabi cross-compiler toolchain.

- Task 1: asgn1_driver - a character device driver exposed through /dev/asgn1 with read, write and IOCTL operations.
- Task 2: counter_driver - a misc device exposed through /dev/counter, using atomic operations and /proc/counter_info for runtime statistics.
- Both modules are built against the same Linux 5.15 kernel configuration with CONFIG_MODVERSIONS enabled.

User Space

```
|-- test_asgn1.sh --> /dev/asgn1 --> asgn1_driver.ko
|-- test_counter.sh --> /dev/counter --> counter_driver.ko
|--> /proc/counter_info
```

Kernel Space: Linux 5.15.0

Hardware Model: ARMv7 Cortex-A9 / QEMU vexpress-a9

```
==== Embedded Linux ASM1 Boot ====
Loading ASM1 ioctl driver...
asgn1_driver: loading out-of-tree module taints kernel.
asgn1: registered with major 241
asgn1: module loaded successfully
asgn1: sysfs attributes created
[ASM1] asgn1_driver loaded successfully
Boot completed! Welcome to Embedded Linux ASM1
```

Figure 1. Successful system boot and automatic loading of asgn1_driver.

2. Task 1 - Character Device Driver with IOCTL

Task 1 implements a character device driver named asgn1_driver. The driver creates /dev/asgn1 and provides standard file operations together with four IOCTL control commands. A mutex protects shared buffer data and statistics so that concurrent accesses do not corrupt driver state.

2.1 Device Operations

The driver provides open(), release(), read(), write(), and unlocked_ioctl(). During the final test, /dev/asgn1 was created as a character device with major number 241. The driver records open_count, write_count, read_count, buffer_len, and last_write_size.

2.2 RESET_BUFFER

RESET_BUFFER clears the internal data buffer and resets read/write-related statistics. The final test confirmed that reading after reset returned an empty result.

2.3 GET_STATS

GET_STATS returns the current driver statistics to user space. The transfer is performed with copy_to_user() so data is safely copied across the user/kernel address-space boundary.

2.4 SET_MODE

The driver supports mode 1 for normal data storage and mode 0 for prefixing written data with [KERNEL]. The test showed that writing hello KERNEL in mode 0 produced [KERNEL]hello KERNEL when read back.

2.5 GET_VERSION and Invalid IOCTL

GET_VERSION returns the string asgn1_driver v1.0. Unsupported IOCTL commands return -ENOTTY; the test program reported INVALID_IOCTL: PASS (-ENOTTY).

```
===== ASM1 TASK 1 TEST =====
[1] Check/Load driver
asgn1_driver 16384 0 - Live 0x7f000000 (0)
[2] Check device
crw----- 1 root 0 241, 0 Oct 7 20:34 /dev/asgn1
[3] Run ioctl test
asgn1: device opened, open_count=1
Device opened successfully
asgn1: ioctl SET_MODE mode=1
SET_MODE = 1
asgn1: ioctl GET_VERSION
driver version = [asgn1_driver v1.0]
asgn1: wrote 10 bytes
write returned = 10
asgn1: read 10 bytes
read data = [hello ASM1]
asgn1: ioctl GET_STATS
open_count = 1
write_count = 1
read_count = 1
buffer_len = 10
last_write_size = 10
asgn1: ioctl RESET_BUFFER
RESET_BUFFER done
asgn1: ioctl SET_MODE mode=0
SET_MODE = 0
asgn1: wrote 12 bytes
write mode 0 returned = 12
asgn1: device released
asgn1: device opened, open_count=2
asgn1: read 20 bytes
read mode 0 data = [[KERNEL]hello KERNEL]
asgn1: ioctl RESET_BUFFER
FINAL RESET_BUFFER done
asgn1: ioctl invalid command
INVALID_IOCTL: PASS (-ENOTTY)
read after reset = []
asgn1: device released
[4] Unload driver
asgn1: module unloaded
[5] Check module after unload
[6] Check device after unload
ls: /dev/asgn1: No such file or directory
===== TEST FINISHED =====
```

Figure 2. Task 1 automated IOCTL test, including version, statistics, reset, and mode switching.

```
SET_MODE = 0
asgn1: wrote 12 bytes
write mode 0 returned = 12
asgn1: device released
asgn1: device opened, open_count=2
asgn1: read 20 bytes
read mode 0 data = [[KERNEL]hello KERNEL]
asgn1: ioctl RESET_BUFFER
FINAL RESET_BUFFER done
asgn1: ioctl invalid command
INVALID_IOCTL: PASS (-ENOTTY)
read after reset = []
asgn1: device released
[4] Unload driver
asgn1: module unloaded
[5] Check module after unload
[6] Check device after unload
ls: /dev/asgn1: No such file or directory
===== TEST FINISHED =====
```

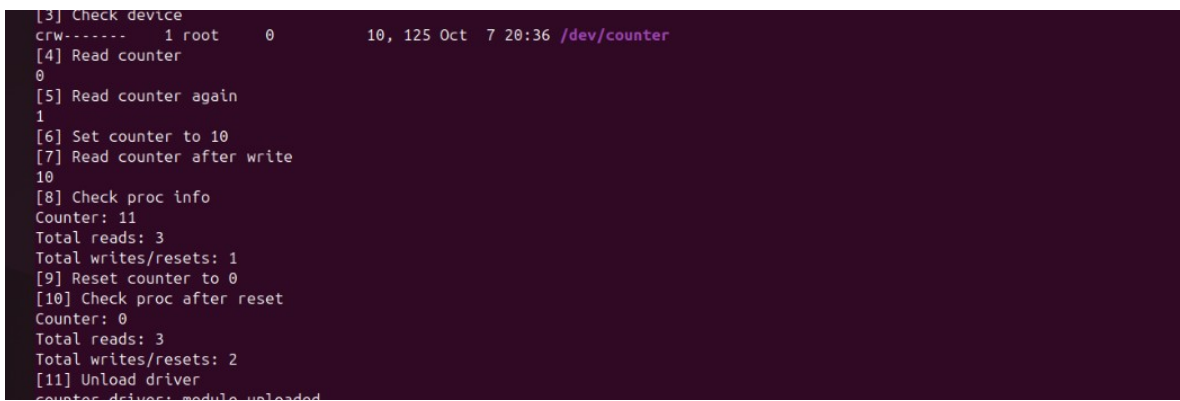
Figure 3. Task 1 final checks: [KERNEL] mode, invalid IOCTL handling, reset, and module unload.

3. Task 2 - Misc Counter Driver

Task 2 implements counter_driver using the Linux miscdevice framework. The device is available at /dev/counter with misc major number 10 and a dynamically assigned minor number. The current counter and the read/write statistics are stored using atomic variables.

3.1 Read and Write Behavior

A read operation returns the current counter value followed by a newline and then increments the counter. Therefore, the first two reads after module load return 0 and 1. A write operation accepts an ASCII integer, parses it with kstrtoint(), and sets the counter to that value. Writing 100 followed by one read returns 100, then the internal counter becomes 101.



```
[3] Check device
crw----- 1 root  0      10, 125 Oct  7 20:36 /dev/counter
[4] Read counter
0
[5] Read counter again
1
[6] Set counter to 10
[7] Read counter after write
10
[8] Check proc info
Counter: 11
Total reads: 3
Total writes/resets: 1
[9] Reset counter to 0
[10] Check proc after reset
Counter: 0
Total reads: 3
Total writes/resets: 2
[11] Unload driver
counter_driver: module unloaded
```

Figure 4. Task 2 automated test: first reads 0 and 1, write 10, next read 10, and procfs statistics.

3.2 /proc/counter_info

The procfs entry /proc/counter_info reports the current counter value, total reads since module load, and total successful writes/resets. Manual testing captured the state before operations, after reads, after writing 100, and after reset.

```

===== TEST FINISHED =====
/ # insmod /usr/lib/modules/counter_driver.ko
counter_driver: module loaded
counter_driver: /proc/counter_info created
/ # echo "===== BEFORE ====="
===== BEFORE =====
/ # cat /proc/counter_info
Counter: 0
Total reads: 0
Total writes/resets: 0
/ #
/ # echo "===== READ 1 ====="
===== READ 1 =====
/ # cat /dev/counter
0
/ #
/ # echo "===== READ 2 ====="
===== READ 2 =====
/ # cat /dev/counter
1
/ #
/ # echo "===== AFTER READS ====="
===== AFTER READS =====
/ # cat /proc/counter_info
Counter: 2
Total reads: 2
Total writes/resets: 0
/ #
/ # echo "===== WRITE 100 ====="
===== WRITE 100 =====
/ # echo 100 > /dev/counter
/ # cat /dev/counter
100
/ #
/ # echo "===== AFTER WRITE ====="
===== AFTER WRITE =====
/ # cat /proc/counter_info
Counter: 101
Total reads: 3
Total writes/resets: 1
/ #
/ # echo "===== RESET ====="
===== RESET =====
/ # echo 0 > /dev/counter
/ # cat /proc/counter_info
Counter: 0
Total reads: 3
Total writes/resets: 2
/ #

```

Figure 5. Manual procs test showing counter and statistics before reads, after reads, after write 100, and after reset.

3.3 Invalid Input Handling

Negative values and non-numeric input are rejected with `-EINVAL`. The shell reports Invalid argument and a non-zero exit code. Invalid writes do not modify the counter and do not increase the successful write/reset statistic.

```

/ # echo "===== INVALID NEGATIVE ====="
===== INVALID NEGATIVE =====
/ # echo -1 > /dev/counter
sh: write error: Invalid argument
/ # echo "exit code = $?"
exit code = 1
/ #
/ # echo "===== INVALID TEXT ====="
===== INVALID TEXT =====
/ # echo abc > /dev/counter
sh: write error: Invalid argument
/ # echo "exit code = $?"
exit code = 1
/ #
/ # echo "===== FINAL PROC ====="
===== FINAL PROC =====
/ # cat /proc/counter_info
Counter: 0
Total reads: 3
Total writes/resets: 2
/ #

```

Figure 6. Invalid negative and text input rejected with Invalid argument; driver state remains unchanged.

4. Comparison of the Two Drivers

Feature	Task 1	Task 2
Driver type	Character device	Misc device
Device node	/dev/asgn1	/dev/counter
Major number	241	10 (misc)
Primary data	Character buffer	Integer counter
Synchronization	Mutex	Atomic operations
IOCTL	Yes - 4 commands	No
Procfs	No	/proc/counter_info
Invalid request	-ENOTTY	-EINVAL

Task 1 demonstrates a traditional character device with custom control operations, while Task 2 demonstrates a simpler miscdevice design with atomic state and procfs monitoring. Together, the two tasks cover different common Linux driver patterns.

5. Testing and Debugging

5.1 Counter Off-by-One Error

The initial counter implementation used `atomic_inc_return()`, so the counter was incremented before the value was returned. The first two reads were therefore 1 and 2 instead of 0 and 1, and writing 10 followed by a read returned 11. The logic was corrected by reading the current value with `atomic_read()` and then incrementing with `atomic_inc()`. The final test returned 0, 1, and 10 as required.

5.2 Missing Runtime Statistics

The initial procfs output showed only the current Counter value. Atomic `read_count` and `write_count` variables were added so `/proc/counter_info` could also display Total reads and Total writes/resets. The final output confirmed that the statistics changed only after successful operations.

5.3 Kernel Module Version Mismatch

A major debugging issue was an "invalid module format" error when loading the rebuilt modules. Investigation showed that the running kernel was based on a configuration with

CONFIG_MODVERSIONS=y, while the current build configuration had MODVERSIONS disabled. The older .config was restored and the Linux kernel was rebuilt so that Module.symvers contained valid CRC values, including a valid module_layout symbol. Both drivers were then rebuilt against that kernel tree.

```
Final vermagic for both modules:  
5.15.0 SMP mod_unload modversions ARMv7 p2v8
```

```
/ # dmesg | grep -E "asgn1|counter_driver"  
asgn1_driver: loading out-of-tree module taints kernel.  
asgn1: registered with major 241  
asgn1: module loaded successfully  
asgn1: sysfs attributes created  
asgn1: device opened, open_count=1  
asgn1: ioctl SET_MODE mode=1  
asgn1: ioctl GET_VERSION  
asgn1: wrote 10 bytes  
asgn1: read 10 bytes  
asgn1: ioctl GET_STATS  
asgn1: ioctl RESET_BUFFER  
asgn1: ioctl SET_MODE mode=0  
asgn1: wrote 12 bytes  
asgn1: device released  
asgn1: device opened, open_count=2  
asgn1: read 20 bytes  
asgn1: ioctl RESET_BUFFER  
asgn1: ioctl invalid command  
asgn1: device released  
asgn1: module unloaded  
counter_driver: module loaded  
counter_driver: /proc/counter_info created  
counter_driver: module unloaded  
counter_driver: module loaded  
counter_driver: /proc/counter_info created  
/ #
```

Figure 7. Kernel messages from both drivers during successful tests.

```
nguyendangquang@nguyendangquang-Bellator:~$ echo "==== TASK 1 ====="  
modinfo -/SE201072_NguyenDangQuang_Asgn01/task1_ioctl/asgn1_driver.ko \  
| grep -E "author|description|license|vermagic"  
  
echo  
echo "==== TASK 2 ====="  
modinfo -/SE201072_NguyenDangQuang_Asgn01/task2_misc/counter_driver.ko \  
| grep -E "author|description|license|vermagic"  
==== TASK 1 =====  
description:   ASM1 Character Device Driver with IOCTL  
author:        Nguyen Dang Quang - SE201072  
license:       GPL  
vermagic:      5.15.0 SMP mod_unload modversions ARMv7 p2v8  
  
==== TASK 2 =====  
description:   ASM1 Counter Misc Device Driver  
author:        Nguyen Dang Quang - SE201072  
license:       GPL  
vermagic:      5.15.0 SMP mod_unload modversions ARMv7 p2v8  
nguyendangquang@nguyendangquang-Bellator:~$
```

Figure 8. Final module metadata and matching vermagic for Task 1 and Task 2.

6. Theory Questions

6.1 Module Reference Count

The module reference count represents active references to a loaded kernel module. It prevents the kernel from unloading a module while code or resources from that module are still in use. For example, an open device file can keep a module referenced. Unloading a module while it is still being used could cause invalid memory accesses or a kernel crash.

6.2 Why copy_to_user() Is Required

Kernel space and user space are protected memory domains. A kernel driver must not treat an arbitrary user pointer as a normal kernel pointer. `copy_to_user()` validates and safely copies data from kernel memory to a user buffer, while `copy_from_user()` performs the reverse transfer. These helpers are necessary because user pages may be invalid or inaccessible and because the MMU enforces address-space protection.

6.3 MTD Device vs. Block Device

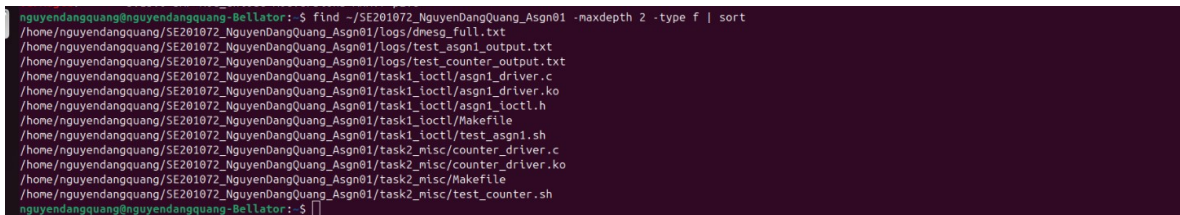
A block device presents storage as addressable blocks and is used for media such as hard disks, SSDs, and SD cards. It integrates with the Linux block layer and commonly hosts filesystems. An MTD (Memory Technology Device) exposes raw flash characteristics such as erase blocks and flash-specific operations. MTD is therefore commonly used for NOR/NAND flash in embedded systems, while block devices provide a conventional block-storage abstraction.

6.4 Purpose of *ppos in read()

*ppos stores the current file position for a file operation. Drivers can use it to determine whether data has already been returned and to provide EOF on a later read. A common pattern is to return 0 when *ppos is greater than zero, then update *ppos after the first successful read. In the counter driver, proper position handling prevents commands such as `cat` from repeatedly reading and incrementing forever during one open operation.

7. Final Submission Evidence

The final submission directory contains only the required source files, kernel modules, test scripts, and test logs. Intermediate build products such as `.o`, `.cmd`, `.mod.c` and `Module.symvers` are not included in the clean submission directory.



```
nguyendangquang@nguyendangquang-Bellator:~$ find -/SE201072_NguyenDangQuang_Asgn01 -maxdepth 2 -type f | sort
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/logs/dmesg_full.txt
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/logs/test_asgn1_output.txt
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/logs/test_counter_output.txt
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task1_ioctl/asgn1_driver.c
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task1_ioctl/asgn1_driver.ko
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task1_ioctl/asgn1_ioctl.h
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task1_ioctl/Makefile
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task1_ioctl/test_asgn1.sh
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task2_misc/counter_driver.c
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task2_misc/counter_driver.ko
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task2_misc/Makefile
/home/nguyendangquang/SE201072_NguyenDangQuang_Asgn01/task2_misc/test_counter.sh
nguyendangquang@nguyendangquang-Bellator:~$
```

Figure 9. Final clean assignment directory structure before adding this report.

```
SE201072_NguyenDangQuang_Asgn01/
|-- task1_ioctl/
|   |-- asgn1_driver.c
|   |-- asgn1_ioctl.h
|   |-- Makefile
|   |-- asgn1_driver.ko
|   `-- test_asgn1.sh
|-- task2_misc/
|   |-- counter_driver.c
|   |-- Makefile
|   |-- counter_driver.ko
|   `-- test_counter.sh
|-- logs/
|   |-- test_asgn1_output.txt
```

```
| |-- test_counter_output.txt  
| `-- dmesg_full.txt  
|-- bao_cao/  
  |-- SE201072_NguyenDangQuang_Asgn01_BaoCao.pdf
```

8. Conclusion

This assignment implemented and tested two Linux kernel drivers on an ARMv7 QEMU platform. Task 1 provided a character device with read/write operations and four IOCTL commands. Task 2 provided a misc counter device with atomic operations, procfs statistics, reset behavior, and input validation. Both modules were rebuilt with kernel-compatible symbol versioning and successfully passed the final functional tests. The debugging process also demonstrated the importance of kernel configuration, Module.symvers CRC data, and vermagic compatibility when developing out-of-tree Linux kernel modules.